scanf(). A compiler is a separate piece of software that reads our program and translates it into a form the machine can execute. Turbo C, GCC and Clang are all compilers; each is one vendor's implementation of the same languages.

An IDE (integrated development environment) is a third thing altogether: the convenient window in which we type the program, with menus and a key to compile and run it. Turbo C++ bundled the compiler and the IDE into a single blue screen, which is why many of us came to think of them as one and the same. They are not. When I recommend 'GCC with Code::Blocks', I am simply recommending a modern compiler (GCC) together with a modern window to use it from (Code::Blocks), the very roles the Turbo compiler and the Turbo editor played earlier.

Why does the standard matter so much? Because it is the common agreement that lets a program written on one computer compile on every other. Whatever the standard defines, every compiler must accept; whatever a single vendor adds on its own is an extension, which works only on that vendor's compiler. Most of the problems described below arise because Turbo C++ stopped following the standard three decades ago, and because its private extensions were mistaken for the language itself.

What the outdated software is costing our students

It predates the language standards themselves: The first official ISO standard for C++ was published in 1998, a full seven years after Turbo C++ 3.0. The 'C++' taught on Turbo is therefore a pre-standard dialect. There is no Standard Template Library (STL), the backbone of modern C++ programming, so students never meet std::vector, std::string, std::map or the standard algorithms such as *sort();* they fall back on raw C-style arrays and character arrays, missing the safety and efficiency the STL provides.

To take one everyday example: to store the marks of a class whose strength is not known in advance, the Turbo student must either declare an oversized array or wrestle with *malloc()* and *realloc()*. The modern student, however, simply declares a std::vector, which grows by itself as elements are added and reports its own size when asked. There are no namespaces, so the line using *namespace std;* that every modern textbook carries is meaningless in the lab. Header files are written as *#include <iostream.h>* instead of the standard *#include <iostream>*. The compiler happily accepts *void main(),* which the standard has never permitted. Exception handling is absent, templates exist only in a primitive early form, and the only casts available are the dangerous C-style casts, with no trace of the safer *static_cast* and *dynamic_cast*.

Since 1998, the language has been revised again and again, through C++11, C++14, C++17, C++20 and C++23, bringing smart pointers (objects that release memory automatically when it is no longer needed, preventing the leaks that plague hand-written new/delete code), auto type deduction (the compiler works out the type of a variable by itself), lambda expressions (small functions written right where they are used), and much else that a Turbo C++ student never even hears of. Modern C++ is almost a different and far more pleasant language. A student who learns C++ only on Turbo has learnt a dialect that no current compiler accepts as it is.

The C language has moved on, but the classroom has not: Turbo C implements roughly the C of 1989 (C89), plus Borland's own extensions. The language has since been revised four times. C99 brought // single-line comments, the freedom to declare variables where they are first needed instead of only at the top of a block, the 64-bit long type, the stdint.h header with exact-width types like int32_t, stdbool.h, inline functions and designated initialisers. C11 added _Static_assert, type-generic programming with _Generic, and a standard threads library. C17 tidied up the standard, and C23, the current edition, has made bool, true and false proper keywords, and added nullptr, binary literals like 0b1010, and much more. Recent releases of GCC already support C23. None of this exists in the Turbo C universe. Students there are studying the C of 1989 and are being told that it is C.

Non-standard libraries taught as if they were the language: This is perhaps the most damaging habit of all. Ask students who have learnt on Turbo C to name the functions they use most often, and *clrscr()* and *getch()* will appear near the top. Both come from *conio.h*, which is not part of the C standard at all. It is a Borland-specific library tied to DOS. The reason is worth spelling out. Functions like *clrscr()* and *getch()* work by calling MS-DOS and the PC's BIOS directly, services that existed on a 1991 computer. Linux and modern Windows are organised entirely differently and offer no such services, so no standard compiler can provide these functions. The C standard deliberately includes only what can be provided on every kind of computer, which is precisely why *conio.h* was never a part of it.

The same trap exists in computer graphics. Institutes still use *graphics.h* to teach the subject, but that is the Borland Graphics Interface (BGI), a proprietary library built for 16-bit MS-DOS; graphics today are handled by standard, cross-platform libraries such as OpenGL, Vulkan and SDL, all freely available. Teaching BGI is like teaching students to drive a horse-drawn carriage in the era of electric cars. The story repeats with *dos.h*, the home of *delay()*. Nothing in the classroom tells students that any of these are non-